Grammar of Graphics and ggplot2

In this multi-part lecture we will be working through an example of building out a nice visualization. We will be learning one of the most common and popular libraries for data visualization in R, **ggplot2**. This lecture will just give a brief introduction to the library and some options for plotting. Eventually we will choose a particular method for static plots and then have separate lectures for each plot type!

Now for a quick overview of ggplot2!

ggplot2 has several advantages:

- Plot specification at a high level of abstraction
- Very flexible
- Theme system for polishing plot appearance
- Mature and complete graphics system
- Many users, active mailing list
- Lot's of online help available (StackOverflow, etc...)

What ggplot2 not ideal for:

- Interactive graphics
- Graph Theory Plots (Graph Nodes)
- 3-D Graphics

Later on we'll learn about other libraries better suited for those topics. As we go through this tutorial on ggplot2, it may be helpful to use this useful cheat sheet for reference when using ggplot2!

ggplot2 also has great documentation! It is most likely you will be referencing either the cheat sheet or the documentation (or these notes) when creating some of your first plots. Don't feel bad if you find yourself referencing them a lot, its a very common practice to go to the documentation, look up what kind of plot you want to make, and then use the skeleton form the documentation to build out your

Grammar of Graphics

ggplot2 is based on the grammar of graphics, the idea that you can build every graph from the same few components: a data set, a set of geoms—visual marks that represent data points, and a coordinate system. To display data values, map variables in the data set to aesthetic properties of the geom like size, color, and x and y locations.

Layers for building Visualizations

ggplot2 is based off the grammar of graphics, which sets a paradigm for data visualization in layers:

We won't go too much in depth to the over all philosophy of the grammar of graphics because the best source of this is from the creator of ggplot, Hadley Wickham, who created a great paper on the topic which you can read here.

As far as the syntax for grammar of graphics and ggplot, we can get a better understanding through some quick examples. In this lecture we'll quickly show some syntax examples, then in the following lectures we'll show various examples of specific plot types using qplot() and ggplot(), then we'll wrap our understanding by building off the final layers of the grammar of graphics and then having an assignment for recreating a plot.

Data and Set-up

Let's get started:

In [1]:

```
# import ggplot2
library(ggplot2)
```

The general syntax of using ggplot2 will look like this:

```
ggplot(data = <default data set>,
       aes(x = <default x axis variable>,
           y = <default y axis variable>,
           ... <other default aesthetic mappings>),
       ... <other plot defaults>) +

       geom_<geom type>(aes(size = <size variable for this geom>,
                            ... <other aesthetic mappings>),
                        data = <data for this point geom>,
                        stat = <statistic string or function>,
                        position = <position string or function>,
                        color = <"fixed color specification">,
                        <other arguments, possibly passed to the _stat_ function) +

  scale_<aesthetic>_<type>(name = <"scale label">,
                           breaks = <where to put tick marks>,
                           labels = <labels for tick marks>,
                           ... <other options for the scale>) +

  theme(plot.background = element_rect(fill = "gray"),
        ... <other theme elements>)
```

We'll build up an understanding of this piece by piece. But first we'll need data! We'll use some real estate data available in this repo or you can download it here

In [3]:

```
library(data.table)
# You may need to put the entire file path to the downloaded csv file!
df <- fread('state_real_estate_data.csv')
```

In [4]:

```
head(df)
```

Out[4]:

	State	region	Date	Home.Value	Structure.Cost	Land.Value	Land.Share..Pct.	Home.Price.Index	Land.Price.Index
1	AK	West	20101	224952	160599	64352	28.6	1.481	1.552
2	AK	West	20102	225511	160252	65259	28.9	1.484	1.576
3	AK	West	20093	225820	163791	62029	27.5	1.486	1.494
4	AK	West	20094	224994	161787	63207	28.1	1.481	1.524
5	AK	West	20074	234590	155400	79190	33.8	1.544	1.885
6	AK	West	20081	233714	157458	76256	32.6	1.538	1.817

In [5]:

```
tail(df)
```

Out[5]:

	State	region	Date	Home.Value	Structure.Cost	Land.Value	Land.Share..Pct.	Home.Price.Index	Land.Price.Index
1	DC	NA	20092	630361	148268	482092	76.5	2.409	2.802
2	DC	NA	20093	632103	148074	484029	76.6	2.415	2.817
3	DC	NA	20114	676463	165456	511007	75.5	2.585	3.025
4	DC	NA	20121	690234	166701	523532	75.8	2.637	3.107
5	DC	NA	20122	705645	167978	537666	76.2	2.696	3.198

	State	region	Date	Home.Value	Structure.Cost	Land.Value	Land.Share..Pct.	Home.Price.Index	Land.Price.Index
6	DC	NA	20123	722914	189289	533625	73.8	2.761	3.299

In [7]:

```
str(df)
```

```
Classes 'data.table' and 'data.frame': 7803 obs. of  9 variables:
 $ State           : chr  "AK" "AK" "AK" "AK" ...
 $ region          : chr  "West" "West" "West" "West" ...
 $ Date            : int  20101 20102 20093 20094 20074 20081 20082 20083 20084 20091 ...
 $ Home.Value      : int  224952 225511 225820 224994 234590 233714 232999 232164 231039 229395 ...
 $ Structure.Cost  : int  160599 160252 163791 161787 155400 157458 160092 162704 164739 165424 ...
 $ Land.Value      : int  64352 65259 62029 63207 79190 76256 72906 69460 66299 63971 ...
 $ Land.Share..Pct.: num  28.6 28.9 27.5 28.1 33.8 32.6 31.3 29.9 28.7 27.9 ...
 $ Home.Price.Index: num  1.48 1.48 1.49 1.48 1.54 ...
 $ Land.Price.Index: num  1.55 1.58 1.49 1.52 1.89 ...
 - attr(*, ".internal.selfref")=<externalptr>
```

In [8]:

```
summary(df)
```

Out[8]:

```
    State              region              Date         Home.Value    
 Length:7803        Length:7803        Min.   :19751   Min.   : 18763  
 Class :character   Class :character   1st Qu.:19843   1st Qu.: 62235  
 Mode  :character   Mode  :character   Median :19941   Median :108724  
                                       Mean   :19939   Mean   :135313  
                                       3rd Qu.:20033   3rd Qu.:172030  
                                       Max.   :20131   Max.   :862885  
 Structure.Cost     Land.Value     Land.Share..Pct. Home.Price.Index
 Min.   : 17825   Min.   :   938   Min.   : 5.00    Min.   :0.1350  
 1st Qu.: 53776   1st Qu.:  4178   1st Qu.: 5.00    1st Qu.:0.4550  
 Median : 88352   Median :  9478   Median :10.40    Median :0.7830  
 Mean   : 99534   Mean   : 35779   Mean   :18.17    Mean   :0.8695  
 3rd Qu.:134871   3rd Qu.: 38631   3rd Qu.:26.30    3rd Qu.:1.2075  
 Max.   :325595   Max.   :594417   Max.   :81.70    Max.   :2.8930  
 Land.Price.Index
 Min.   : 0.0000  
 1st Qu.: 0.0020  
 Median : 0.2520  
 Mean   : 0.9912  
 3rd Qu.: 1.1510  
 Max.   :15.4340  
```

Using ggplot2

Quick Example with Histograms

Histograms are a great way of quickly exploring your data! We have a couple of options for quickly producing histograms off the columns of a data frame. We have:

- hist()
- qplot()
- ggplot()

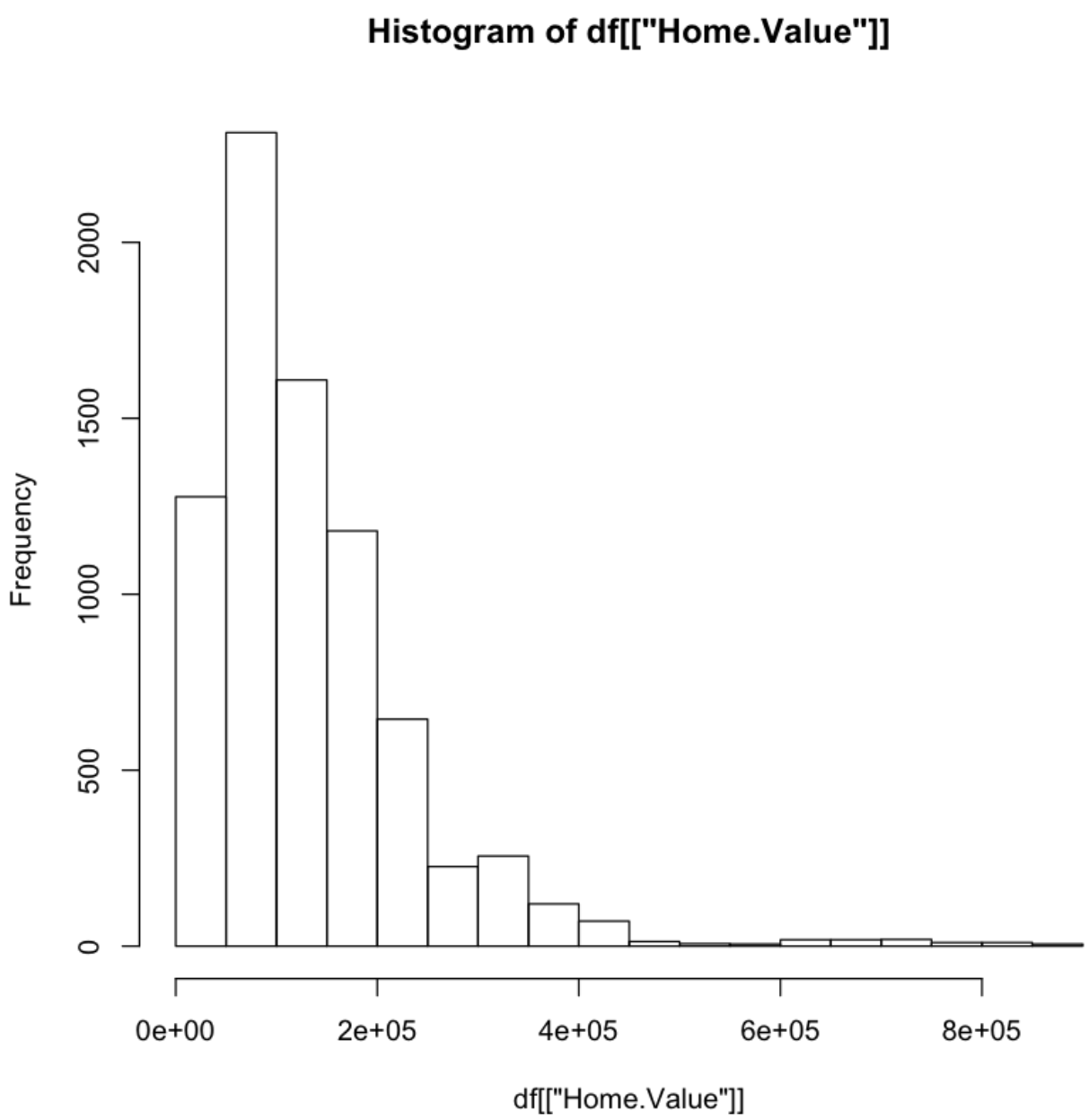
They differ mainly in one main component, for each of these methods you usually trade-off ease of use for ability to customize.

Note! In RStudio you'll need to call print(plot_name) to display your plots. Also the plots will look a lot better in RStudio than here in the notes.

Let's show quick use cases of each:

In [12]:

```
# Pass a column straight into hist()
hist(df[['Home.Value']])
```



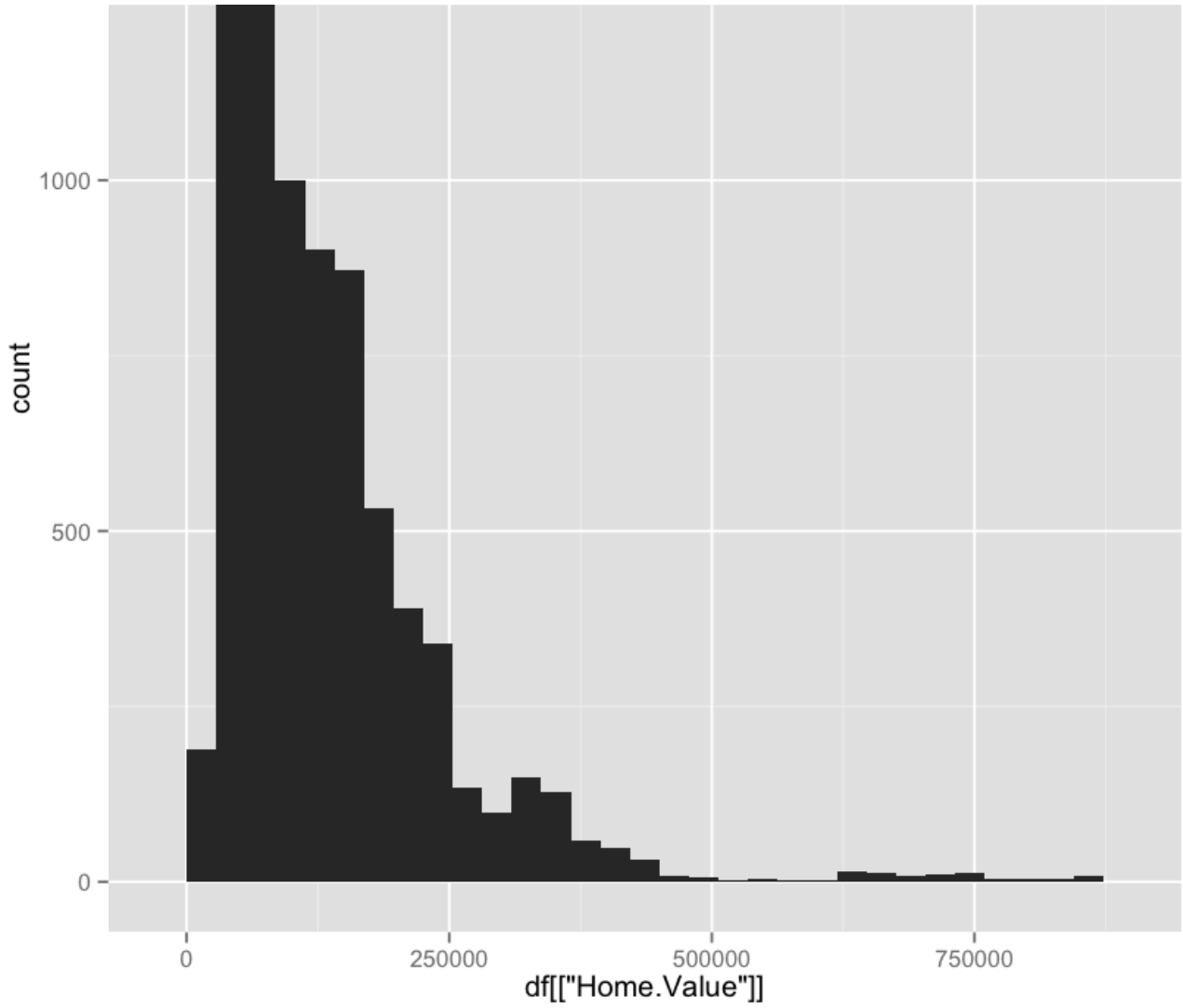
Using qplot

Notice the auto-adjustment of the color theme and the binwidth.

In [14]:

```
qplot(df[['Home.Value']])
```

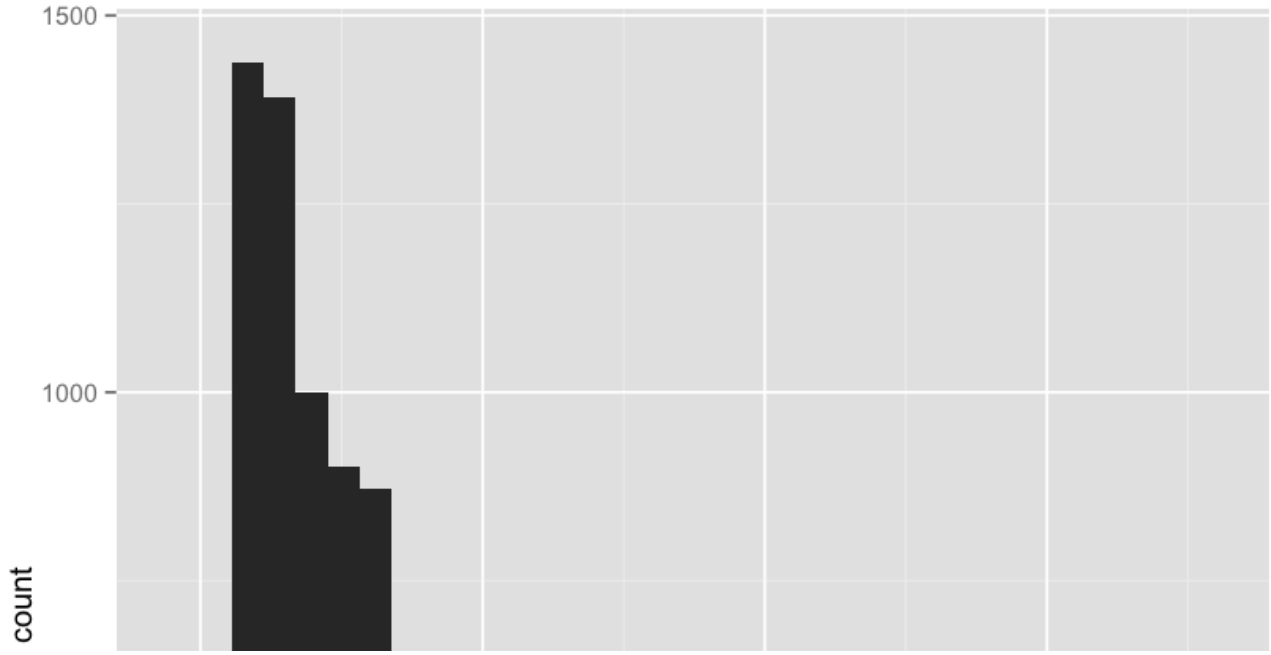
```
stat_bin: binwidth defaulted to range/30. Use 'binwidth = x' to adjust this.
```

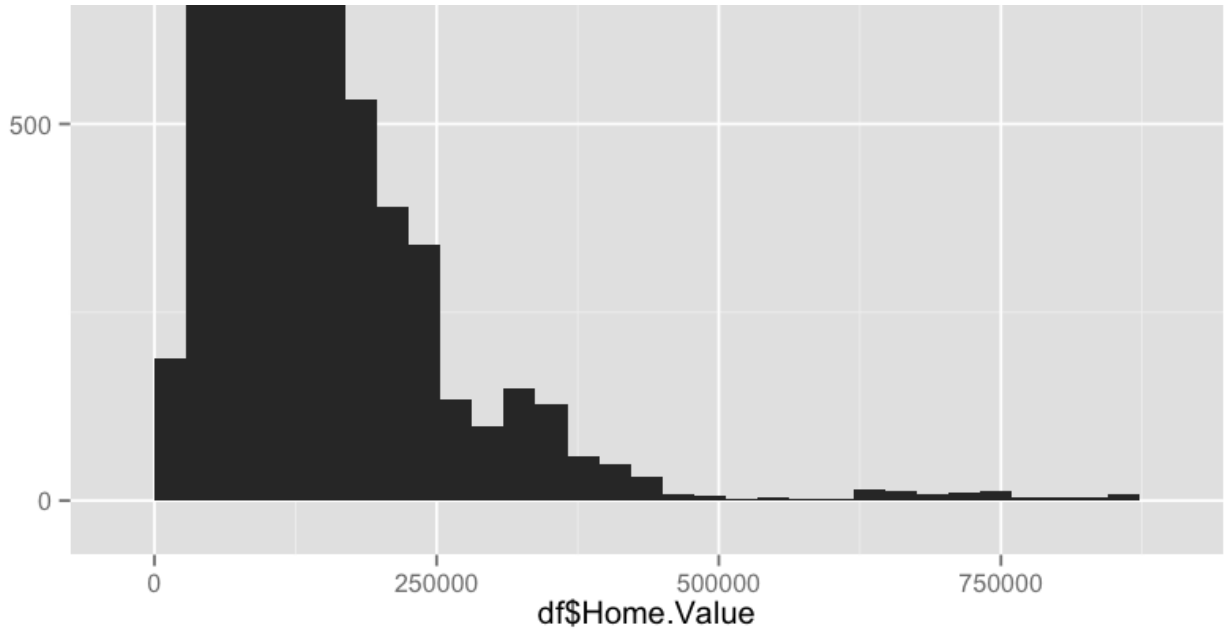


In [19]:

```
# Using ggplot, lots of ability to customize, but bit more complicated!
ggplot(data = df,aes(df$Home.Value))+geom_histogram()
```

```
stat_bin: binwidth defaulted to range/30. Use 'binwidth = x' to adjust this.
```





So what method should we choose? Usually the **qplot()** function will give us a nice balance between ease of use and ability to customize, let's quickly break down the syntax for using **qplot()**.

qplot

The qplot() function can be used to create the most common graph types. While it does not expose ggplot's full power, it can create a very wide range of useful plots. The format is:

```
qplot(x, y, data=, color=, shape=, size=, alpha=, geom=, method=, formula=, facets=, xlim=, ylim
= xlab=, ylab=, main=, sub=)
```

Each of these additional arguments provide methods for customizing your plot further:

option	description
alpha	Alpha transparency for overlapping elements expressed as a fraction between 0 (complete transparency) and 1 (complete opacity)
color, shape, size, fill	Associates the levels of variable with symbol color, shape, or size. For line plots, color associates levels of a variable with line color. For density and box plots, fill associates fill colors with a variable. Legends are drawn automatically.
data	Specifies a data frame
facets	Creates a trellis graph by specifying conditioning variables. Its value is expressed as *rowvar ~ colvar*. To create trellis graphs based on a single conditioning variable, use *rowvar~*. or *.~colvar*)
geom	Specifies the geometric objects that define the graph type. The geom option is expressed as a character vector with one or more entries. geom values include "point", "smooth", "boxplot", "line", "histogram", "density", "bar", and "jitter".
main, sub	Character vectors specifying the title and subtitle
method, formula	If geom="smooth", a loess fit line and confidence limits are added by default. When the number of observations is greater than 1,000, a more efficient smoothing algorithm is employed. Methods include "lm" for regression, "gam" for generalized additive models, and "rlm" for robust regression. The formula parameter gives the form of the fit. For example, to add simple linear regression lines, you'd specify geom="smooth", method="lm", formula=y~x. Changing the formula to y~poly(x,2) would produce a quadratic fit. Note that the formula uses the letters x and y, not the names of the variables. For method="gam", be sure to load the mgcv package. For method="rml", load the MASS package.
x, *y*	Specifies the variables placed on the horizontal and vertical axis. For univariate plots (for example, histograms), omit *y*
xlab, ylab	Character vectors specifying horizontal and vertical axis labels

xlim,ylim	Two-element numeric vectors giving the minimum and maximum values for the horizontal and vertical axes, respectively

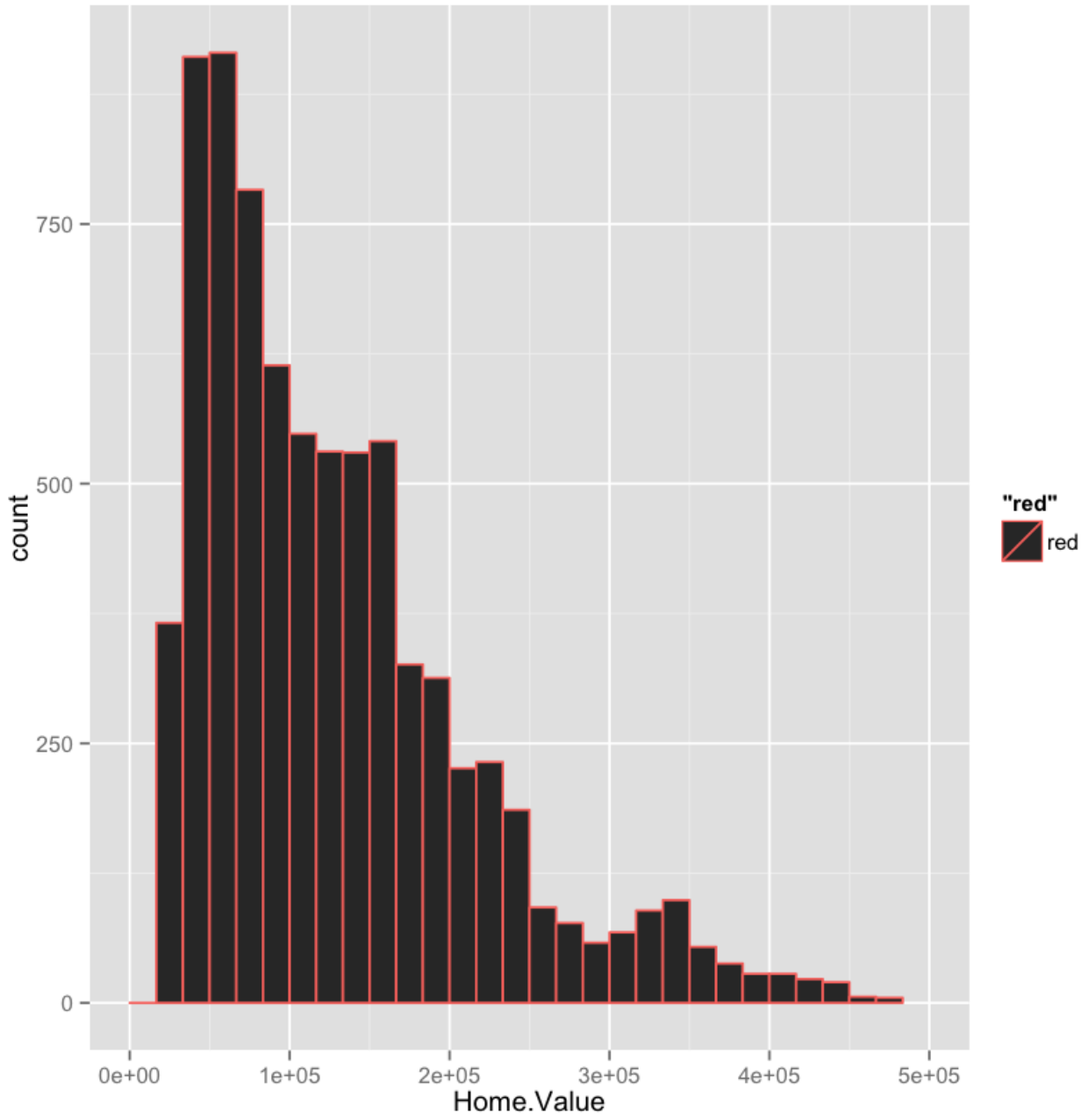
Quick Example of Customization

Let's explore qplot further! In the last example we just passed a single column and qplot automatically knew to do a histogram, from now on we're going to be a little more formal and pass in the entire data source and then specify what columns to grab and how to plot it:

In [34]:

```
# Customize the histogram further
qplot(data=df,x=Home.Value,geom = 'histogram',xlim=c(0,500000), color='red')
```

```
stat_bin: binwidth defaulted to range/30. Use 'binwidth = x' to adjust this.
```



Conclusion

Great! Hopefully you've begin to see how powerful ggplot2. From now on we will explore each plot type indvidually and show how to construct it in **qplot** and then show how to create it with **ggplot()**. This ggplot knowledge will be especially useful when we begin to create interactive visualizations with plotly's library.

create interactive visualizations with plotly's library.